

Practical 11

In [25]: # — Product Cipher (Rail Fence + Row Transposition)

```
# Stage 1: Rail Fence Cipher
def rail_fence_encrypt(text, rails):
    fence = [[] for _ in range(rails)]
    rail, direction = 0, 1

    for char in text:
        fence[rail].append(char)
        if rail == 0:            direction = 1
        if rail == rails - 1:    direction = -1
        rail += direction

    return ''.join(''.join(row) for row in fence)

def rail_fence_decrypt(text, rails):
    n = len(text)
    pattern, rail, direction = [], 0, 1

    for _ in range(n):
        pattern.append(rail)
        if rail == 0:            direction = 1
        if rail == rails - 1:    direction = -1
        rail += direction

    indices = sorted(range(n), key=lambda i: pattern[i])
    result = [''] * n
    for i, char in zip(indices, text):
        result[i] = char
    return ''.join(result)
```

In [26]: # Stage 2: Row Transposition Cipher

```
def row_encrypt(text, key):
    cols = len(key)
    rows = -(-len(text) // cols)
    text = text.ljust(rows * cols, 'x')

    grid = [list(text[i*cols:(i+1)*cols]) for i in range(rows)]
    order = sorted(range(cols), key=lambda i: key[i])

    return ''.join(''.join(grid[r][c] for r in range(rows)) for c in order)

def row_decrypt(text, key):
    cols = len(key)
    rows = len(text) // cols
    order = sorted(range(cols), key=lambda i: key[i])

    grid, idx = {}, 0
    for c in order:
```

```

        grid[c] = list(text[idx:idx+rows])
        idx += rows

    return ''.join(grid[c][r] for r in range(rows) for c in range(cols))

```

```

In [27]: # Stage 3: Product Cipher = Rail Fence → Row Transposition
def product_encrypt(text, rails, key):
    stage1 = rail_fence_encrypt(text, rails) # Step 1: Rail Fence
    stage2 = row_encrypt(stage1, key)         # Step 2: Row Transposition
    return stage1, stage2

def product_decrypt(text, rails, key):
    stage1 = row_decrypt(text, key)           # Reverse Step 2
    stage2 = rail_fence_decrypt(stage1, rails) # Reverse Step 1
    return stage1, stage2

```

```

In [28]: # — Demo —————
msg      = "HELLOWORLD"
rails    = 3
key      = [3, 1, 4, 2]

s1, enc  = product_encrypt(msg, rails, key)
r1, dec  = product_decrypt(enc, rails, key)

print("=== Product Cipher ===")
print(f"Original      : {msg}")
print(f"After Rail Fence: {s1}")
print(f"After Row Trans : {enc}")
print(f"--- Decryption ---")
print(f"After Row Trans : {r1}")
print(f"After Rail Fence: {dec}")

```

```

=== Product Cipher ===
Original      : HELLOWORLD
After Rail Fence: HOLELWRDLO
After Row Trans : OWOEDxHLLLRx
--- Decryption ---
After Row Trans : HOLELWRDLOxx
After Rail Fence: HEOLOWxRLDxL

```